



Diffusion Models Course

Learn Diffusion Models from scratch

By Hugging Face

github.com/huggingface/diffusion-models-class

Hugging Face Diffusion Models Course

In this free course, you will:

- 🎓 Study the theory behind diffusion models
- 🚀 Learn how to generate images and audio with the popular 😊 Diffusers library
- 🧑🔬 Train your own diffusion models from scratch
- 📺 Fine-tune existing diffusion models on new datasets
- 🌐 Explore conditional generation and guidance
- 🧑🔬 Create your own custom diffusion model pipelines

Prerequisites

This course requires a good level in Python and a grounding in deep learning and Pytorch.

If it's not the case yet, you can check these free resources:

- Python: <https://www.udacity.com/course/introduction-to-python--ud1110>
- Intro to Deep Learning with PyTorch: <https://www.udacity.com/course/deep-learning-pytorch--ud188>
- PyTorch in 60min: https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html

To upload your models to the Hugging Face Hub, you'll need an account. You can create one for free at the following address: <https://huggingface.co/join>.

What is the syllabus?

The course consists in four units. Each unit is made up of a theory section, which also lists resources/papers, and two *notebooks*. More specifically, we have:

- Unit 1: Introduction to diffusion models

Introduction to 😊 Diffusers and implementation from 0

- Unit 2: Finetuning and guidance

Finetuning a diffusion model on new data and adding guidance.

- Unit 3: Stable Diffusion

Exploring a powerful text-conditioned latent diffusion model

- Unit 4: Doing more with diffusion

Advanced techniques for going further with diffusion

Who are we?

About the authors:

Jonathan Whitaker is a Data Scientist/AI Researcher doing R&D with answer.ai. He likes teaching and making courses. His current focus is on generative AI, flitting between several modalities. You can find more info at: johnnowhitaker.dev.

Lewis Tunstall is a machine learning engineer at Hugging Face, focused on developing open-source tools and making them accessible to the wider community. He is also a co-author of the O'Reilly book [Natural Language Processing with Transformers](#).

FAQ

Here are some answers to frequently asked questions:

- **Does taking this course lead to a certification?**

Currently we do not have any certification for this course. However, we are working on a certification program for the Hugging Face ecosystem -- stay tuned!

- **How much time should I spend on this course?**

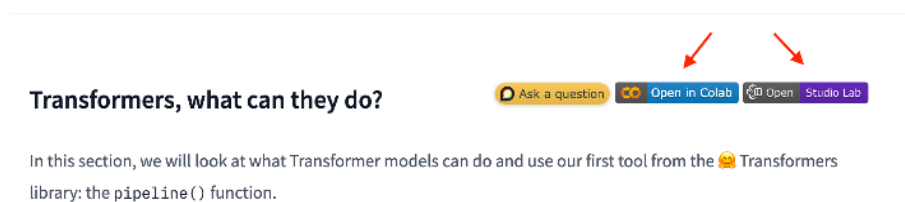
Each chapter in this course is designed to be completed in 1 week, with approximately 6-8 hours of work per week. However, you can take as much time as you need to complete the course.

- **Where can I ask a question if I have one?**

If you have a question about any section of the course, just click on the "Ask a question" banner at the top of the page to be automatically redirected to the right section of the [Hugging Face Discord](#) to ask your question in the channel `#diffusion-models-class`.

- **Where can I get the code for the course?**

For each section, click on the banner at the top of the page to run the code:



- **How can I contribute to the course?**

There are many ways to contribute to the course! If you find a typo or a bug, please open an issue on the `diffusion-models-class` repo.

If you would like to help translate the course into your native language, check out the instructions [here](#).

- **Can I reuse this course?**

Of course! The course is released under the permissive [Apache 2 license](#). This means that you must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use. If you would like to cite the course, please use the following BibTeX:

```
@misc{huggingfacecourse,  
  author = {Hugging Face},  
  title = {The Hugging Face Diffusion Models Course, 2022},  
  howpublished = "\url{https://huggingface.co/course}",  
  year = {2022},  
  note = "[Online; accessed <today>]"  
}
```

Let's get started!

Are you ready to get started? Then go to the first unit to start the course.

Unit 1: An Introduction to Diffusion Models

Welcome to Unit 1 of the Hugging Face Diffusion Models Course! In this unit, you will learn the basics of how diffusion models work and how to create your own using the 🧑🏻‍🔬 Diffusers library.

Start this Unit 🚀

Here are the steps for this unit:

- Make sure you've [signed up for this course](#) so that you can be notified when new material is released.
- Read through the introductory material below as well as any of the additional resources that sound interesting.
- Check out the ***Introduction to Diffusers*** notebook below to put theory into practice with the 🧑🏻‍🔬 Diffusers library.
- Train and share your own diffusion model using the notebook or the linked training script.
- (Optional) Dive deeper with the ***Diffusion Models from Scratch*** notebook if you're interested in seeing a minimal from-scratch implementation and exploring the different design decisions involved.
- (Optional) Check out [this video](#) for an informal run-through the material for this unit.

🗣️ Don't forget to join the [Discord](#), where you can discuss the material and share what you've made in the `#diffusion-models-class` channel.

What Are Diffusion Models?

Diffusion models are a relatively recent addition to a group of algorithms known as 'generative models'. The goal of generative modeling is to learn to **generate** data, such as images or audio, given a number of training examples. A good generative model will create a **diverse** set of outputs that resemble the training data without being exact copies. How do diffusion models achieve this? Let's focus on the image generation case for illustrative purposes.

diffusion-course-lgjqiw9a/images/img_1.svg)](https://kaggle.com/kernels/welcome?src=https://github.com/huggingface/diffusion-models-class/blob/main/unit1/01_introduction_to_diffusers.ipynb) |  ([Gradient])(/tmp/hf-diffusion-course-lgjqiw9a/images/img_2.svg)]

(https://console.paperspace.com/github/huggingface/diffusion-models-class/blob/main/unit1/01_introduction_to_diffusers.ipynb) |  ([Open In SageMaker Studio Lab](https://studiolab.sagemaker.aws/studiolab.svg))

(https://studiolab.sagemaker.aws/import/github/huggingface/diffusion-models-class/blob/main/unit1/01_introduction_to_diffusers.ipynb) | | Diffusion Models from Scratch |  ([Open In Colab])(/tmp/hf-diffusion-course-lgjqiw9a/images/img_0.svg)]

(https://colab.research.google.com/github/huggingface/diffusion-models-class/blob/main/unit1/02_diffusion_models_from_scratch.ipynb) |  ([Kaggle])(/tmp/hf-diffusion-course-lgjqiw9a/images/img_1.svg)](https://kaggle.com/kernels/welcome?src=https://github.com/huggingface/diffusion-models-class/blob/main/unit1/02_diffusion_models_from_scratch.ipynb) |  ([Gradient])(/tmp/hf-diffusion-course-lgjqiw9a/images/img_2.svg)]

(https://console.paperspace.com/github/huggingface/diffusion-models-class/blob/main/unit1/02_diffusion_models_from_scratch.ipynb) |  ([Open In SageMaker Studio Lab](https://studiolab.sagemaker.aws/studiolab.svg))

(https://studiolab.sagemaker.aws/import/github/huggingface/diffusion-models-class/blob/main/unit1/02_diffusion_models_from_scratch.ipynb) | In **Introduction to Diffusers**, we show the different steps described above using building blocks from the diffusers library. You'll quickly see how to create, train and sample your own diffusion models on whatever data you choose. By the end of the notebook, you'll be able to read and modify the example training script to train diffusion models and share them with the world! This notebook also introduces the main exercise associated with this unit, where we will collectively attempt to figure out good 'training recipes' for diffusion models at different scales - see the next section for more info. In **Diffusion Models from Scratch**, we show those same steps (adding noise to data, creating a model, training and sampling) but implemented from scratch in PyTorch as simply as possible. Then we compare this 'toy example' with the diffusers version, noting how the two differ and where improvements have been made. The goal here is to gain familiarity with the different components and the design decisions that go into them so that when you look at a new implementation you can quickly identify the key ideas. **Project Time** Now that you've got the basics down, have a go at training one or more diffusion models! Some suggestions are included at the end of the **Introduction to Diffusers** notebook. Make sure to share your results, training recipes and findings with the community so that we can collectively figure out the best ways to train these models. **Some Additional Resources** [The Annotated Diffusion Model]

(<https://huggingface.co/blog/annotated-diffusion>) is a very in-depth walk-through of the code and theory behind DDPMs with maths and code showing all the different components. It also links to a number of papers for further reading. Hugging Face documentation on [Unconditional Image-Generation] (https://huggingface.co/docs/diffusers/training/unconditional_training) for some examples of how to train diffusion models using the official training example script, including code showing how to create your own dataset. AI Coffee Break video on Diffusion Models: <https://www.youtube.com/watch?v=344w5h24-h8> Yannic Kilcher Video on DDPMs: <https://www.youtube.com/watch?v=W-O7AZNzbzQ> Found more great resources? Let us know and we'll add them to this list.

Unit 2: Fine-Tuning, Guidance and Conditioning

Welcome to Unit 2 of the Hugging Face Diffusion Models Course! In this unit, you will learn how to use and adapt pre-trained diffusion models in new ways. You will also see how we can create diffusion models that take additional inputs as **conditioning** to control the generation process.

Start this Unit

Here are the steps for this unit:

- Make sure you've [signed up for this course](#) so that you can be notified when new material is released.
- Read through the material below for an overview of the key ideas of this unit.
- Check out the ***Fine-tuning and Guidance*** notebook to fine-tune an existing diffusion model on a new dataset using the  Diffusers library and to modify the sampling procedure using guidance.
- Follow the example in the notebook to share a Gradio demo for your custom model.
- (Optional) Check out the ***Class-conditioned Diffusion Model Example*** notebook to see how we can add additional control to the generation process.
- (Optional) Check out [this video](#) for an informal run-through of the material in this unit.

 Don't forget to join the [Discord](#), where you can discuss the material and share what you've made in the `#diffusion-models-class` channel.

Fine-Tuning

As you may have seen in Unit 1, training diffusion models from scratch can be time-consuming! Especially as we push to higher resolutions, the time and data required to train a model from scratch can become impractical. Fortunately, there is a solution: begin with a model that has already been trained! This way we start from a model that has already learned to denoise images of some kind, and the hope is that this provides a better starting point than beginning from a randomly initialized model.



Fine-tuning typically works best if the new data somewhat resembles the base model's original training data (for example, beginning with a model trained on faces is probably a good idea if you're trying to generate cartoon faces) but surprisingly the benefits persist even if the domain is changed quite drastically. The image above is generated from a [model trained on the LSUN Bedrooms dataset](#) and fine-tuned for 500 steps on the [WikiArt dataset](#). The [training script](#) is included for reference alongside the notebooks for this unit.

Guidance

Unconditional models don't give much control over what is generated. We can train a conditional model (more on that in the next section) that takes additional inputs to help steer the generation process, but what if we already have a trained unconditional model we'd like to use? Enter guidance, a process by which the model predictions at each step in the generation process are evaluated against some guidance function and modified such that the final generated image is more to our liking.

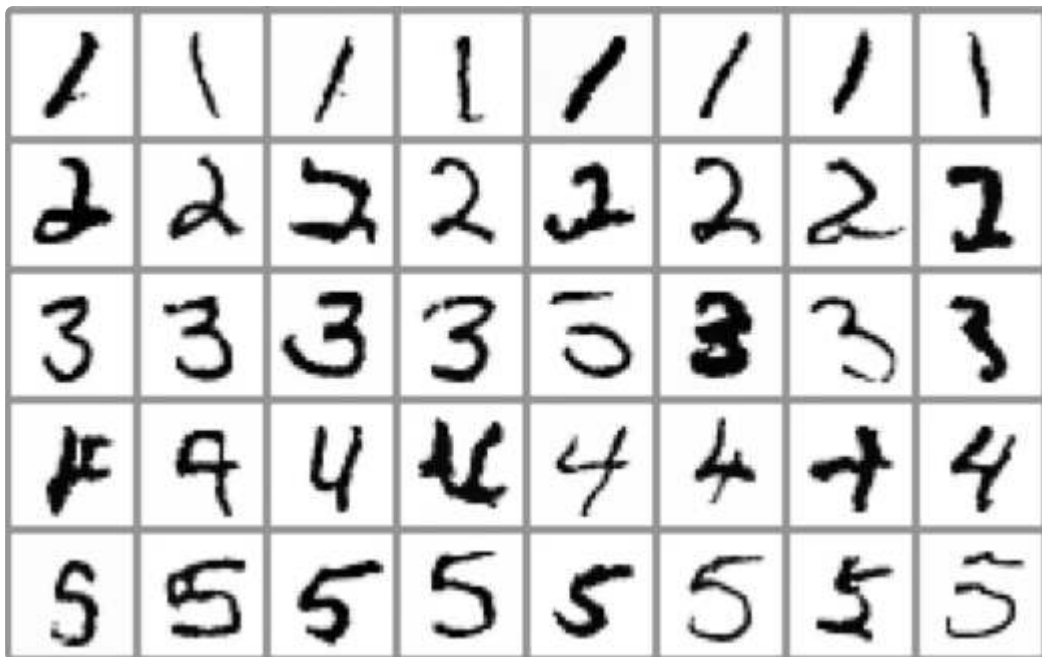


This guidance function can be almost anything, making this a powerful technique! In the notebook, we build up from a simple example (controlling the color, as illustrated in the example output above) to one utilizing a powerful pre-trained model called CLIP which lets us

guide generation based on a text description.

Conditioning

Guidance is a great way to get some additional mileage from an unconditional diffusion model, but if we have additional information (such as a class label or an image caption) available during training then we can also feed this to the model for it to use as it makes its predictions. In doing so, we create a **conditional** model, which we can control at inference time by controlling what is fed in as conditioning. The notebook shows an example of a class-conditioned model which learns to generate images according to a class label.



There are a number of ways to pass in this conditioning information, such as

- Feeding it in as additional channels in the input to the UNet. This is often used when the conditioning information is the same shape as the image, such as a segmentation mask, a depth map or a blurry version of the image (in the case of a restoration/superresolution model). It does work for other types of conditioning too. For example, in the notebook, the class label is mapped to an embedding and then expanded to be the same width and height as the input image so that it can be fed in as additional channels.
- Creating an embedding and then projecting it down to a size that matches the number of channels at the output of one or more internal layers of the UNet, and then adding it to those outputs. This is how the timestep conditioning is handled, for example. The output of each Resnet block has a projected timestep embedding added to it. This is useful when you have a vector such as a CLIP image embedding as your conditioning information. A notable example is the ['Image Variations' version of Stable Diffusion](#) which does exactly this.
- Adding cross-attention layers that can 'attend' to a sequence passed in as conditioning. This

is most useful when the conditioning is in the form of some text - the text is mapped to a sequence of embeddings using a transformer model, and then cross-attention layers in the UNet are used to incorporate this information into the denoising path. We'll see this in action in Unit 3 as we examine how Stable Diffusion handles text conditioning.

Hands-On Notebook

Chapter	Colab	Kaggle	Gradient	Studio Lab
Fine-tuning and Guidance	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab
Class-conditioned Diffusion Model Example	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab

At this point, you know enough to get started with the accompanying notebooks! Open them in your platform of choice using the links above. Fine-tuning is quite computationally intensive, so if you're using Kaggle or Google Colab make sure you set the runtime type to 'GPU' for the best results.

The bulk of the material is in ***Fine-tuning and Guidance***, where we explore these two topics through worked examples. The notebook shows how you can fine-tune an existing model on new data, add guidance, and share the result as a Gradio demo. There is an accompanying script ([finetune_model.py](#)) that makes it easy to experiment with different fine-tuning settings, and an [example space](#) that you can use as a template for sharing your own demo on 😊 Spaces.

In the ***Class-conditioned Diffusion Model Example***, we show a brief worked example of creating a diffusion model conditioned on class labels using the MNIST dataset. The focus is on demonstrating the core idea as simply as possible: by giving the model extra information about what it is supposed to be denoising, we can later control what kinds of images are generated at inference time.

Project Time

Following the examples in the ***Fine-tuning and Guidance*** notebook, fine-tune your own model or pick an existing model and create a Gradio demo to showcase your new guidance skills. Don't forget to share your demo on Discord, Twitter etc so we can admire your work!

Some Additional Resources

[Denoising Diffusion Implicit Models](#) - Introduced the DDIM sampling method (used by DDIMScheduler)

[GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models](#) - Introduced methods for conditioning diffusion models on text

[eDiffi: Text-to-Image Diffusion Models with an Ensemble of Expert Denoisers](#) - Shows how many different kinds of conditioning can be used together to give even more control over the kinds of samples generated

Found more great resources? Let us know and we'll add them to this list.

Unit 3: Stable Diffusion

Welcome to Unit 3 of the Hugging Face Diffusion Models Course! In this unit you will meet a powerful diffusion model called Stable Diffusion (SD) and explore what it can do.

Start this Unit 🚀

Here are the steps for this unit:

- Make sure you've [signed up for this course](#) so that you can be notified when new material is released.
- Read through the material below for an overview of the key ideas of this unit.
- Check out the [Stable Diffusion Introduction notebook](#) to see SD applied in practice to some common use-cases.
- Use the **Dreambooth** notebook in the [hackathon folder](#) to fine-tune your own custom Stable Diffusion model and share it with the community for a chance to win some prizes and swag.
- (Optional) Check out the [Stable Diffusion Deep Dive video](#) and the accompanying [notebook](#) for a deeper exploration of the different components and how they can be adapted for different effects. This material was created for the new FastAI course, 'Stable Diffusion from the Foundations' - the whole course has been released, making this a great supplement to this class for anyone curious about building these kinds of models completely from scratch.

🗨️ Don't forget to join the [Discord](#), where you can discuss the material and share what you've made in the `#diffusion-models-class` channel.

Introduction



Example images generated using Stable Diffusion

Stable Diffusion is a powerful text-conditioned latent diffusion model. Don't worry, we'll explain those words shortly! Its ability to create amazing images from text descriptions has made it an internet sensation. In this unit, we're going to explore how SD works and see what other tricks it can do.

Latent Diffusion

As image size grows, so does the computational power required to work with those images. This is especially pronounced in an operation called self-attention, where the amount of operations grows quadratically with the number of inputs. A 128px square image has 4x as many pixels as a 64px square image, and so requires 16x (i.e. 4^2) the memory and compute in a self-attention layer. This is a problem for anyone who'd like to generate high-resolution images!

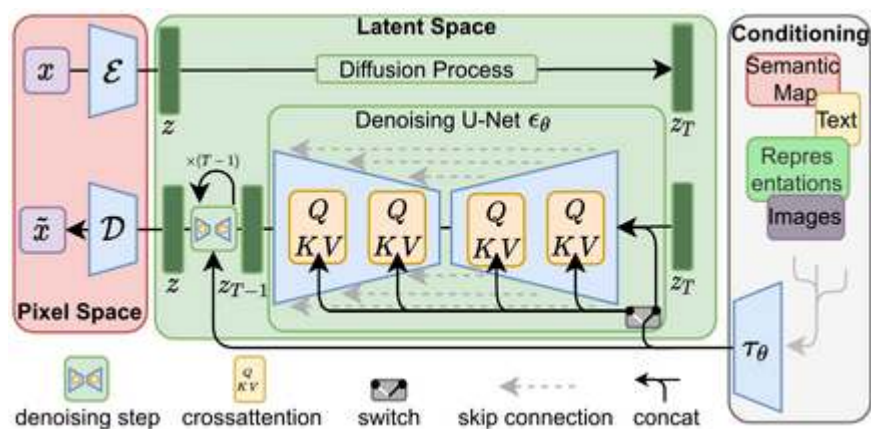


Diagram from the [Latent Diffusion paper](#)

Latent diffusion helps to mitigate this issue by using a separate model called a Variational Auto-Encoder (VAE) to **compress** images to a smaller spatial dimension. The rationale behind this is that images tend to contain a large amount of redundant information - given enough training data, a VAE can hopefully learn to produce a much smaller representation of an input image and then reconstruct the image based on this small **latent** representation with a high degree of fidelity. The VAE used in SD takes in 3-channel images and produces a 4-channel latent representation with a reduction factor of 8 for each spatial dimension. That is, a 512px square input image will be compressed down to a 4x64x64 latent.

By applying the diffusion process on these **latent representations** rather than on full-resolution images, we can get many of the benefits that would come from using smaller images (lower memory usage, fewer layers needed in the UNet, faster generation times...) and still decode the result back to a high-resolution image once we're ready to view the final result. This innovation dramatically lowers the cost to train and run these models.

Text Conditioning

In Unit 2 we showed how feeding additional information to the UNet allows us to have some additional control over the types of images generated. We call this conditioning. Given a noisy version of an image, the model is tasked with predicting the denoised version **based on additional clues** such as a class label or, in the case of Stable Diffusion, a text description of the image. At inference time, we can feed in the description of an image we'd like to see and some pure noise as a starting point, and the model does its best to 'denoise' the random input into something that matches the caption.

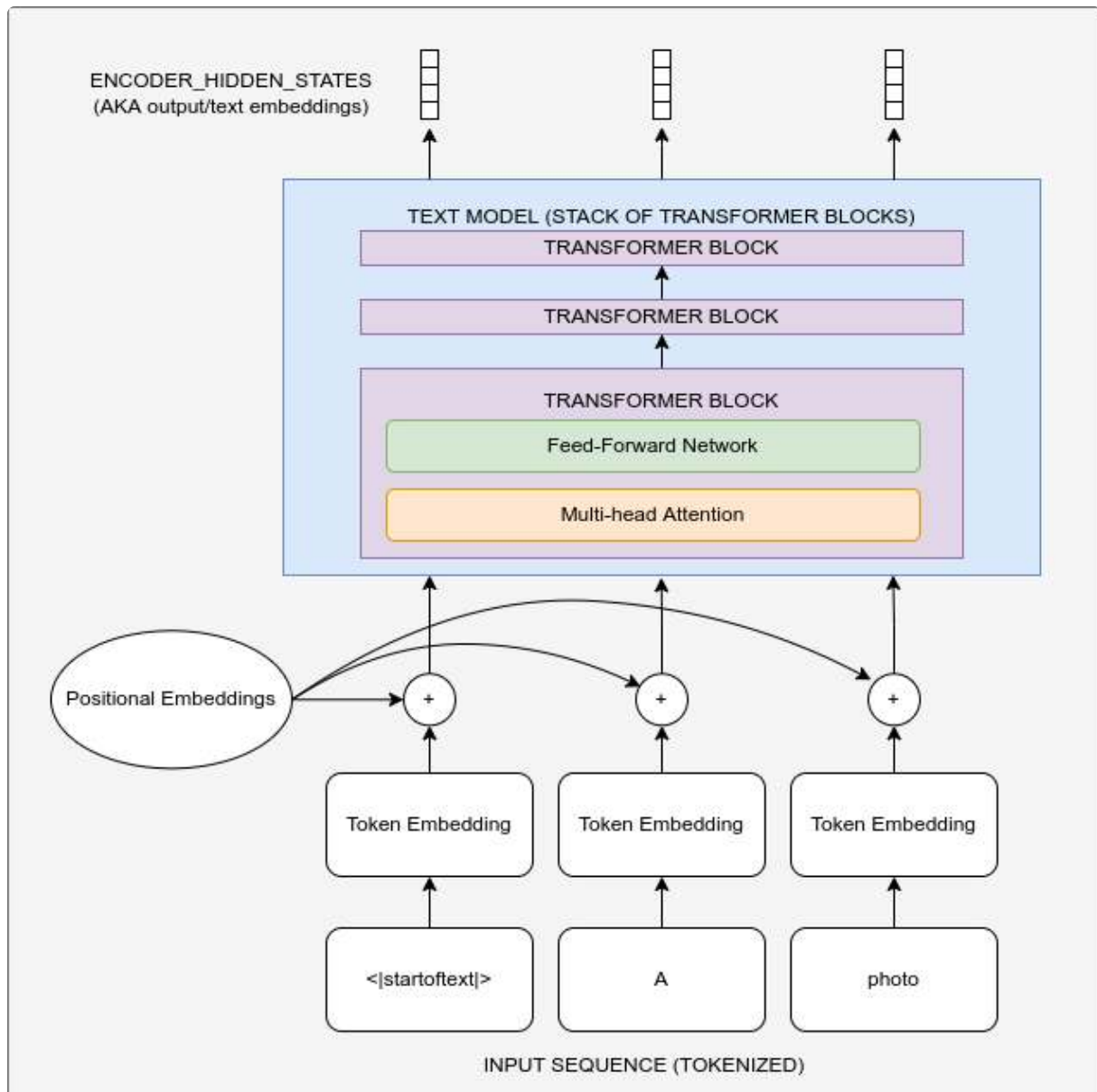
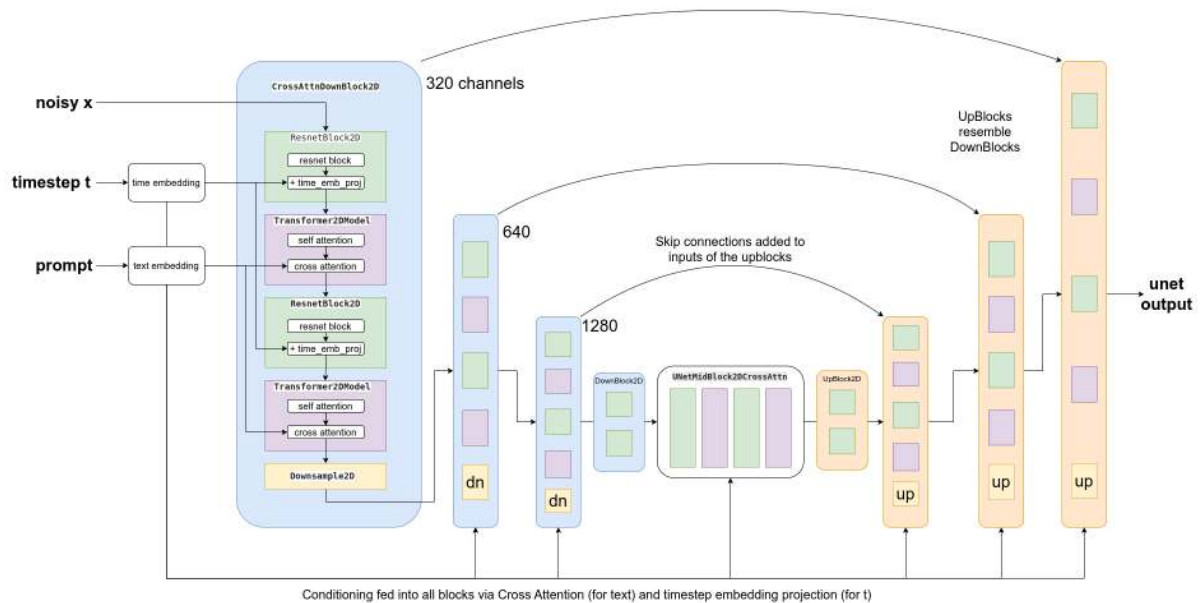


Diagram showing the text encoding process which transforms the input prompt into a set of text embeddings (the `encoder_hidden_states`) which can then be fed in as conditioning to the UNet.

For this to work, we need to create a numeric representation of the text that captures relevant information about what it describes. To do this, SD leverages a pre-trained transformer model based on something called CLIP. CLIP's text encoder was designed to process image captions into a form that could be used to compare images and text, so it is well suited to the task of creating useful representations from image descriptions. An input prompt is first tokenized (based on a large vocabulary where each word or sub-word is assigned a specific token) and then fed through the CLIP text encoder, producing a 768-dimensional (in the case

of SD 1.X) or 1024-dimensional (SD 2.X) vector for each token. To keep things consistent prompts are always padded/truncated to be 77 tokens long, and so the final representation which we use as conditioning is a tensor of shape 77x1024 per prompt.



OK, so how do we actually feed this conditioning information into the UNet for it to use as it makes predictions? The answer is something called cross-attention. Scattered throughout the UNet are cross-attention layers. Each spatial location in the UNet can 'attend' to different tokens in the text conditioning, bringing in relevant information from the prompt. The diagram above shows how this text conditioning (as well as timestep-based conditioning) is fed in at different points. As you can see, at every level the UNet has ample opportunity to make use of this conditioning!

Classifier-free Guidance

It turns out that even with all of the effort put into making the text conditioning as useful as possible, the model still tends to default to relying mostly on the noisy input image rather than the prompt when making its predictions. In a way, this makes sense - many captions are only loosely related to their associated images and so the model learns not to rely too heavily on the descriptions! However, this is undesirable when it comes time to generate new images - if the model doesn't follow the prompt then we may get images out that don't relate to our description at all.



Images generated from the prompt "An oil painting of a collie in a top hat" with CFG scale 0, 1, 2 and 10 (left to right)

To fix this, we use a trick called Classifier-Free Guidance (CFG). During training, text conditioning is sometimes kept blank, forcing the model to learn to denoise images with no text information whatsoever (unconditional generation). Then at inference time, we make two separate predictions: one with the text prompt as conditioning and one without. We can then use the difference between these two predictions to create a final combined prediction that pushes **even further** in the direction indicated by the text-conditioned prediction according to some scaling factor (the guidance scale), hopefully resulting in an image that better matches the prompt. The image above shows the outputs for a prompt at different guidance scales - as you can see, higher values result in images that better match the description.

Other Types of Conditioning: Super-Resolution, Inpainting and Depth-to-Image

It is possible to create versions of Stable Diffusion that take in additional kinds of conditioning. For example, the [Depth-to-Image model](#) has extra input channels that take in-depth information about the image being denoised, and at inference time we can feed in the depth map of a target image (estimated using a separate model) to hopefully generate an image with a similar overall structure.

depth to image example

Depth-conditioned SD is able to generate different images with the same overall structure (example from StabilityAI)

In a similar manner, we can feed in a low-resolution image as the conditioning and have the model generate the high-resolution version (as used by the [Stable Diffusion Upscaler](#)). Finally, we can feed in a mask showing a region of the image to be re-generated as part of the 'in-painting' task, where the non-mask regions need to stay intact while new content is generated for the masked area.

Fine-Tuning with DreamBooth



Image from the [dreambooth project page](#) based on the Imagen model

DreamBooth is a technique for fine-tuning a text-to-image model to 'teach' it a new concept, such as a specific object or style. The technique was originally developed for Google's Imagen model but was quickly adapted to [work for stable diffusion](#). Results can be extremely impressive (if you've seen anyone with an AI profile picture on social media recently the odds are high it came from a dreambooth-based service) but the technique is also sensitive to the settings used, so check out our notebook and [this great investigation into the different training parameters](#) for some tips on getting it working as well as possible.

Hands-On Notebook

Chapter	Colab	Kaggle	Gradient	Studio Lab
Stable Diffusion Introduction	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab
DreamBooth Hackathon Notebook	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab
Stable Diffusion Deep Dive	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab

At this point, you know enough to get started with the accompanying notebooks! Open them in your platform of choice using the links above. DreamBooth requires quite a lot of compute power, so if you're using Kaggle or Google Colab make sure you set the runtime type to 'GPU' for the best results.

The 'Stable Diffusion Introduction' notebook is a short introduction to stable diffusion with the 🤖 Diffusers library, stepping through some basic usage examples using pipelines to generate and modify images.

In the DreamBooth Hackathon Notebook (in the [hackathon folder](#)) we show how you can fine-tune SD on your own images to create a custom version of the model covering a new style or concept.

Finally, the 'Stable Diffusion Deep Dive' notebook and video break down every step in a typical generation pipeline, suggesting some novel ways to modify each stage for additional creative control.

Project Time

Follow the instructions in the **DreamBooth** notebook to train your own model for one of the specified categories. Make sure you include the example outputs in your submission so that we can choose the best models in each category! See the [hackathon info](#) for details on prizes, GPU credits and more.

Some Additional Resources

- [High-Resolution Image Synthesis with Latent Diffusion Models](#) - The paper that introduced the approach behind Stable Diffusion.
- [CLIP](#) - CLIP learns to connect text with images and the CLIP text encoder is used to transform a text prompt into the rich numerical representation used by SD. See also, [this article on OpenCLIP](#) for some background on recent open-source CLIP variants (one of which is used for SD version 2).
- [GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models](#) an early paper demonstrating text conditioning and CFG.

Found more great resources? Let us know and we'll add them to this list.

Unit 4: Going Further with Diffusion Models

Welcome to Unit 4 of the Hugging Face Diffusion Models Course! In this unit, we will look at some of the many improvements and extensions to diffusion models appearing in the latest research. It will be less code-heavy than previous units have been and is designed to give you a jumping-off point for further research.

Start this Unit

Here are the steps for this unit:

- Make sure you've [signed up for this course](#) so that you can be notified when additional units are added to the course.
- Read through the material below for an overview of the different topics covered in this unit.
- Dive deeper into any specific topics with the linked videos and resources.
- Explore the demo notebooks and then read the 'What Next' section for some project suggestions.

 Don't forget to join [Discord](#), where you can discuss the material and share what you've made in the `#diffusion-models-class` channel.

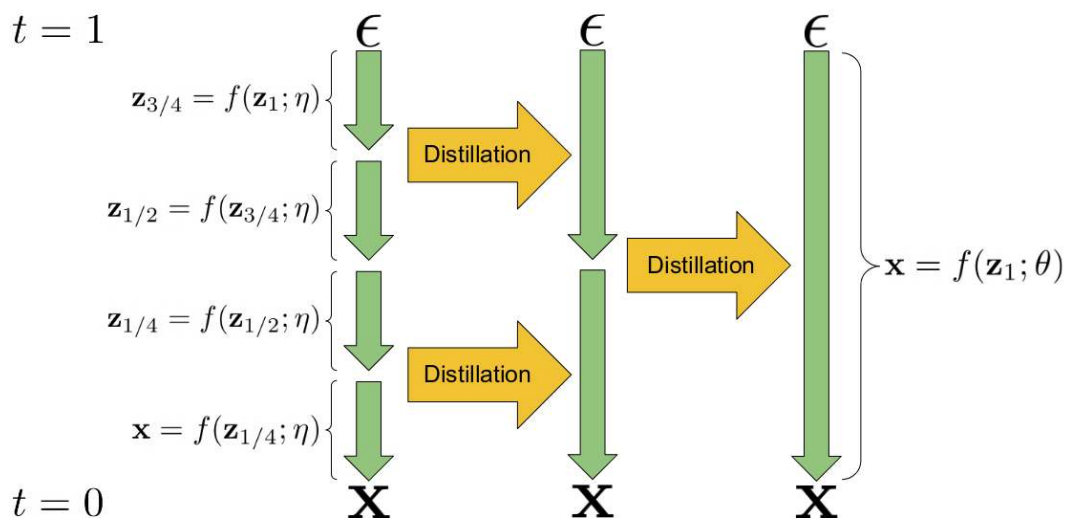
Table of Contents

- [Unit 4: Going Further with Diffusion Models](#)
- [Start this Unit !\[\]\(9b53d87d01ca4eb10fdff342d0db74ef_img.jpg\)](#)
- [Table of Contents](#)
- [Faster Sampling via Distillation](#)
- [Training Improvements](#)
- [More Control for Generation and Editing](#)
- [Video](#)
- [Audio](#)
- [New Architectures and Approaches - Towards 'Iterative Refinement'](#)

- [Hands-On Notebooks](#)
- [Where Next?](#)

Faster Sampling via Distillation

Progressive distillation is a technique for taking an existing diffusion model and using it to train a new version of the model that requires fewer steps for inference. The 'student' model is initialized from the weights of the 'teacher' model. During training, the teacher model performs two sampling steps and the student model tries to match the resulting prediction in a single step. This process can be repeated multiple times, with the previous iteration's student model becoming the teacher for the next stage. The result is a model that can produce decent samples in much fewer steps (typically 4 or 8) than the original teacher model. The core mechanism is illustrated in this diagram from the [paper that introduced the idea](#):



Progressive Distillation illustrated (from the [paper](#))

The idea of using an existing model to 'teach' a new model can be extended to create guided models where the classifier-free guidance technique is used by the teacher model and the student model must learn to produce an equivalent output in a single step based on an additional input specifying the targeted guidance scale. This further reduces the number of model evaluations required to produce high-quality samples. [This video](#) gives an overview of the approach.

NB: A distilled version of Stable Diffusion can be used [here](#).

Key references:

- [Progressive Distillation For Fast Sampling Of Diffusion Models](#)
- [On Distillation Of Guided Diffusion Models](#)

Training Improvements

There have been several additional tricks developed to improve diffusion model training. In this section we've tried to capture the core ideas from recent papers. There is a constant stream of research coming out with additional improvements, so if you see a paper you feel should be added here please let us know!

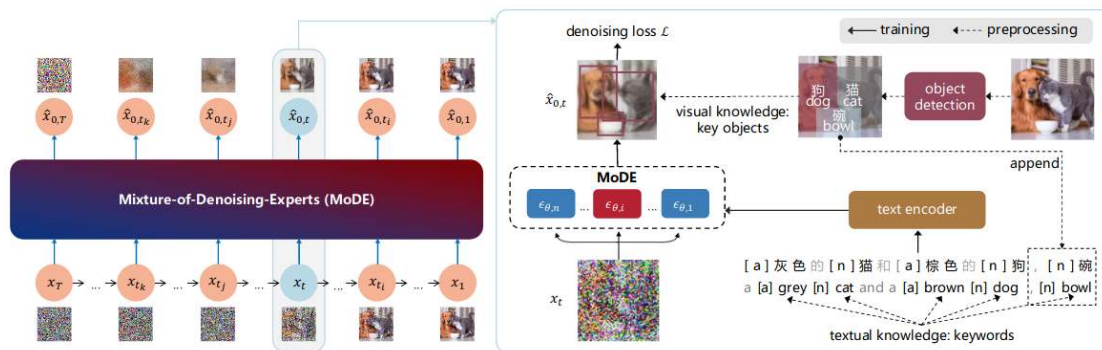


Figure 2: The illustration of ERNIE-ViLG 2.0 model architecture.

Figure 2 from the [ERNIE-ViLG 2.0 paper](#)

Key training improvements:

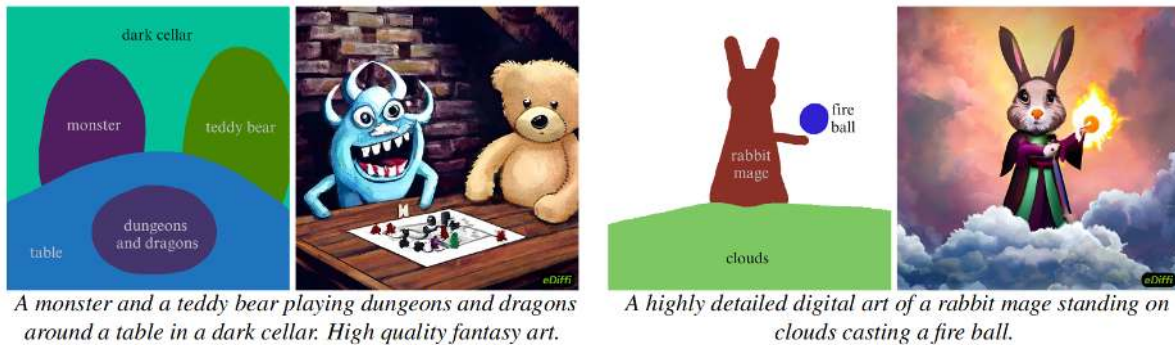
- Tuning the noise schedule, loss weighting and sampling trajectories for more efficient training. An excellent paper exploring some of these design choices is [Elucidating the Design Space of Diffusion-Based Generative Models](#) by Karras et al.
- Training on diverse aspect ratios, as described in [this video from the course launch event](#).
- Cascaded diffusion models, training one model at low resolution and then one or more super-res models. Used in DALLÉ-2, Imagen and more for high-resolution image generation.
- Better conditioning, incorporating rich text embeddings (Imagen uses a large language model called T5) or multiple types of conditioning (eDiffi).
- 'Knowledge Enhancement' - incorporating pre-trained image captioning and object detection models into the training process to create more informative captions and produce better performance ([ERNIE-ViLG 2.0](#)).
- 'Mixture of Denoising Experts' (MoDE) - training different variants of the model ('experts') for different noise levels as illustrated in the image above from the [ERNIE-ViLG 2.0 paper](#).

Key references:

- [Elucidating the Design Space of Diffusion-Based Generative Models](#)
- [eDiffi: Text-to-Image Diffusion Models with an Ensemble of Expert Denoisers](#)
- [ERNIE-ViLG 2.0: Improving Text-to-Image Diffusion Model with Knowledge-Enhanced Mixture-of-Denoising-Experts](#)
- [Imagen - Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding](#) ([demo site](#))

More Control for Generation and Editing

In addition to training improvements, there have been several innovations in the sampling and inference phase, including many approaches that can add new capabilities to existing diffusion models.



Samples generated by 'paint-with-words' ([eDiffi](#))

The video '[Editing Images with Diffusion Models](#)' gives an overview of the different methods being used to edit existing images with diffusion models. The available techniques can be split into four main categories:

- 1) Add noise and then denoise with a new prompt. This is the idea behind the [img2img](#) pipeline, which has been modified and extended in various papers:
 - [SDEdit](#) and [MagicMix](#) build on this idea.
 - DDIM inversion ([TODO link tutorial](#)) uses the model to 'reverse' the sampling trajectory rather than adding random noise, giving more control.
 - [Null-text Inversion](#) enhances the performance of this kind of approach dramatically by optimizing the unconditional text embeddings used for classifier-free guidance at each step, allowing for extremely high-quality text-based image editing.
- 2) Extending the ideas in (1) but with a mask to control where the effect is applied:
 - [Blended Diffusion](#) introduces the basic idea.
 - [This demo](#) uses an existing segmentation model (CLIPSeg) to create the mask based on a

text description.

- [DiffEdit](#) is an excellent paper that shows how the diffusion model itself can be used to generate an appropriate mask for editing the image based on text.

- [SmartBrush: Text and Shape Guided Object Inpainting with Diffusion Model](#) fine-tunes a diffusion model for more accurate mask-guided inpainting.

3) Cross-attention Control: using the cross-attention mechanism in diffusion models to control the spatial location of edits for more fine-grained control:

- [Prompt-to-Prompt Image Editing with Cross Attention Control](#) is the key paper that introduced this idea, and the technique has since been applied to [Stable Diffusion](#).

- This idea is also used for 'paint-with-words' ([eDiffi](#), shown above).

4) Fine-tune ('overfit') on a single image and then generate with the fine-tuned model. The following papers both published variants of this idea at roughly the same time:

- [Imagic: Text-Based Real Image Editing with Diffusion Models](#).

- [UniTune: Text-Driven Image Editing by Fine Tuning an Image Generation Model on a Single Image](#)

.

The paper [InstructPix2Pix: Learning to Follow Image Editing Instructions](#) is notable in that it used some of the image editing techniques described above to build a synthetic dataset of image pairs alongside image edit instructions (generated with GPT3.5) to train a new model capable of editing images based on natural language instructions.

Video



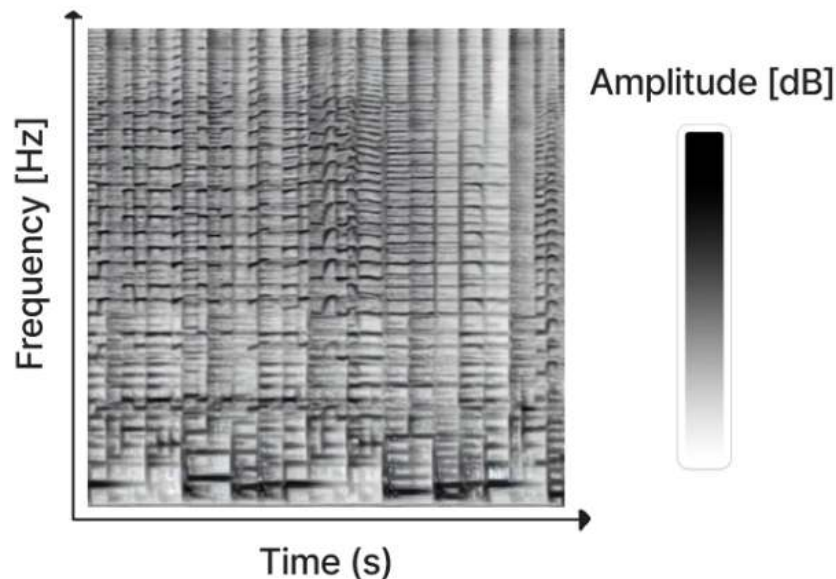
Still frames from *sample videos generated with Imagen Video*

A video can be represented as a sequence of images, and the core ideas of diffusion models can be applied to these sequences. Recent work has focused on finding appropriate architectures (such as '3D UNets' which operate on entire sequences) and on working efficiently with video data. Since high-frame-rate video involves a lot more data than still images, current approaches tend to first generate low-resolution and low-frame-rate video and then apply spatial and temporal super-resolution to produce the final high-quality video outputs.

Key references:

- [Video Diffusion Models](#)
- [IMAGEN VIDEO: HIGH DEFINITION VIDEO GENERATION WITH DIFFUSION MODELS](#)

Audio



A spectrogram generated with Riffusion ([image source](#))

While there has been some work on generating audio directly using diffusion models (e.g. [DiffWave](#)) the most successful approach so far has been to convert the audio signal into something called a spectrogram, which effectively 'encodes' the audio as a 2D "image" which can then be used to train the kinds of diffusion models we're used to using for image generation. The resulting generated spectrograms can then be converted into audio using existing methods. This approach is behind the recently-released Riffusion, which fine-tuned Stable Diffusion to generate spectrograms conditioned on text - [try it out here](#).

The field of audio generation is moving extremely quickly. Over the past week (at the time of writing) there have been at least 5 new advances announced, which are marked with a star in the list below:

Key references:

- [DiffWave: A Versatile Diffusion Model for Audio Synthesis](#)
- 'Riffusion' (and [code](#))
- *MusicLM* by Google generates consistent audio from text and can be conditioned with hummed or whistled melodies.
- [RAVE2](#) - A new version of a Variational Auto-Encoder that will be useful for latent diffusion on audio tasks. This is used in the soon-to-be-announced *AudioLDM* model.
- [Noise2Music](#) - A diffusion model trained to produce high-quality 30-second clips of audio based on text descriptions.
- *Make-An-Audio: Text-To-Audio Generation with Prompt-Enhanced Diffusion Models* - A diffusion model trained to generate diverse sounds based on text.
- [Moûsai: Text-to-Music Generation with Long-Context Latent Diffusion](#)

New Architectures and Approaches - Towards 'Iterative Refinement'

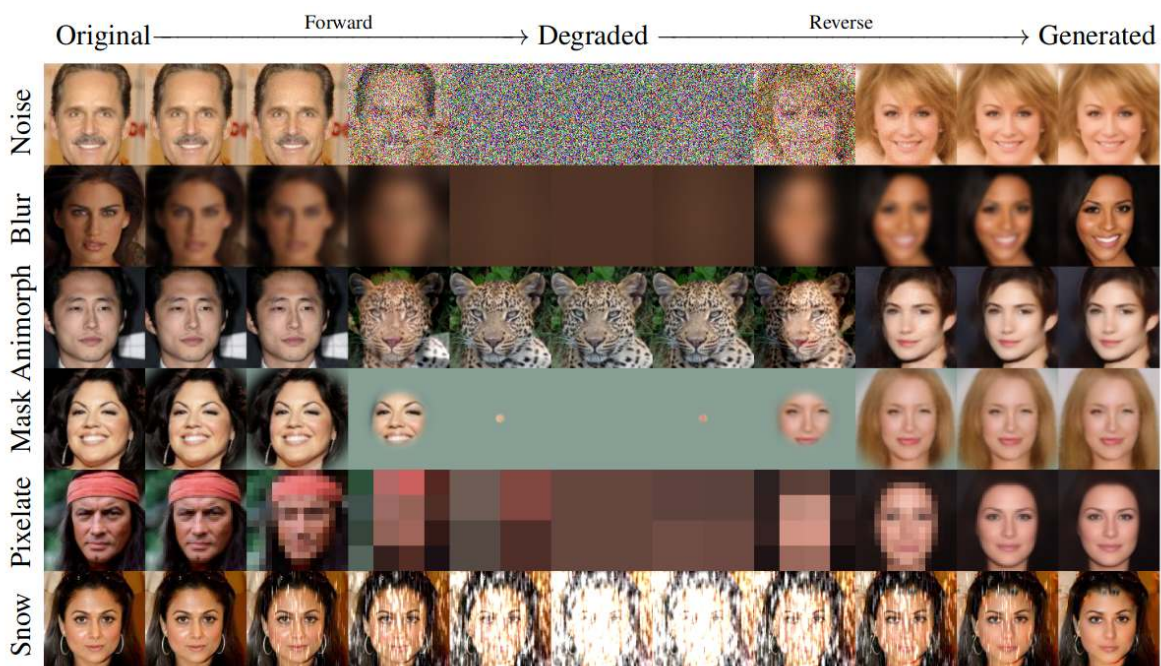
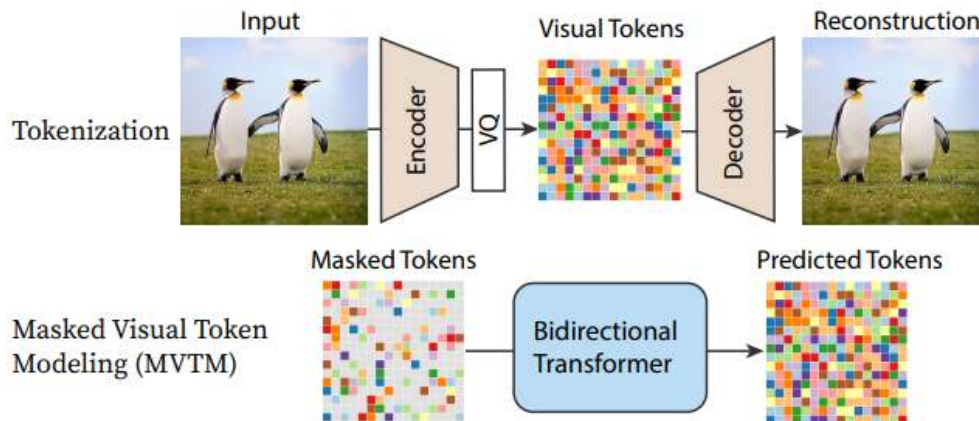


Figure 1: Demonstration of the forward and backward processes for both hot and cold diffusions. While standard diffusions are built on Gaussian noise (top row), we show that generative models can be built on arbitrary and even noiseless/cold image transforms, including the ImageNet-C *snowification* operator, and an *animorphosis* operator that adds a random animal image from AFHQ.

Figure 1 from the [Cold Diffusion](#) paper

We are slowly moving beyond the original narrow definition of "diffusion" models and towards a more general class of models that perform **iterative refinement**, where some form of corruption (like the addition of gaussian noise in the forward diffusion process) is gradually

reversed to generate samples. The 'Cold Diffusion' paper demonstrated that many other types of corruption can be iteratively 'undone' to generate images (examples shown above), and recent transformer-based approaches have demonstrated the effectiveness of token replacement or masking as a noising strategy.



Pipeline from [MaskGIT](#)

The UNet architecture at the heart of many current diffusion models is also being replaced with different alternatives, most notably various transformer-based architectures. In [Scalable Diffusion Models with Transformers \(DiT\)](#) a transformer is used in place of the UNet for a fairly standard diffusion model approach, with excellent results. [Recurrent Interface Networks](#) applies a novel transformer-based architecture and training strategy in pursuit of additional efficiency. [MaskGIT](#) and [MUSE](#) use transformer models to work with tokenized representations of images, although the [Paella](#) model demonstrates that a UNet can also be applied successfully to these token-based regimes too.

With each new paper, more efficient approaches are being developed, and it may be some time before we see what peak performance looks like on these kinds of iterative refinement tasks. There is much more still to explore!

Key references

- [Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise](#)
- [Scalable Diffusion Models with Transformers \(DiT\)](#)
- [MaskGIT: Masked Generative Image Transformer](#)
- [Muse: Text-To-Image Generation via Masked Generative Transformers](#)
- [Fast Text-Conditional Discrete Denoising on Vector-Quantized Latent Spaces \(Paella\)](#)
- [Recurrent Interface Networks](#) - A promising new architecture that does well at generating high-resolution images without relying on latent diffusion or super-resolution. See also, [simple diffusion: End-to-end diffusion for high-resolution images](#) which highlights the importance of the noise schedule for training at higher resolutions.

Hands-On Notebooks

Chapter	Colab	Kaggle	Gradient	Studio Lab
DDIM Inversion	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab
Diffusion for Audio	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab

We've covered a LOT of different ideas in this unit, many of which deserve much more detailed follow-on lessons in the future. For now, you can choose two of the many topics via the hands-on notebooks we've prepared.

- **DDIM Inversion** shows how a technique called inversion can be used to edit images using existing diffusion models.
- **Diffusion for Audio** introduces the idea of spectrograms and shows a minimal example of fine-tuning an audio diffusion model on a specific genre of music.

Where Next?

This is the final unit of this course for now, which means that what comes next is up to you! Remember that you can always ask questions and chat about your projects on the Hugging Face [Discord](#). We look forward to seeing what you create 🤖.

DreamBooth Hackathon

🚩 The hackathon is now over and the winners have been announced on Discord. You are still welcome to train models and submit them to the leaderboard, but we won't be offering prizes or certificates at this point in time.

Welcome to the DreamBooth Hackathon! This is a community event where you'll **personalise a Stable Diffusion model by fine-tuning it on a handful of your own images**. To do so, you'll use a powerful technique called *DreamBooth*, which allows one to implant a subject (e.g. your pet or favourite dish) into the output domain of the model such that it can be synthesized with a *unique identifier* in the prompt.

This competition is composed of 5 *themes*, where each theme will collect models belong to the following categories:

- **Animal** 🐾: Use this theme to generate images of your pet or favourite animal hanging out in the Acropolis, swimming, or flying in space.
- **Science** 🔬: Use this theme to generate cool synthetic images of galaxies, proteins, or any domain of the natural and medical sciences.
- **Food** 🍔: Use this theme to tune Stable Diffusion on your favourite dish or cuisine.
- **Landscape** 🏞️: Use this theme to generate beautiful landscapes of your favourite mountain, lake, or garden.
- **Wildcard** 🔥: Use this theme to go wild and create Stable Diffusion models for any category of your choosing!

We'll be **giving out prizes to the top 3 most liked models per theme**, and you're encouraged to submit as many models as you want!

Getting started

Follow the steps below to take part in this event:

1. Join the [Hugging Face Discord server](#) and check out the `#dreambooth-hackathon` channel to stay up to date with the event.
2. Launch and run the [DreamBooth notebook](#) to train your models by clicking on one of the links below. Make sure you select the GPU runtime in each platform to ensure your models train fast!

Notebook	Colab	Kaggle	Gradient	Studio Lab
DreamBooth Training	 Open in Colab	 Open in Kaggle	 Run on Gradient	 Open In SageMaker Studio Lab

Note 🙌: The DreamBooth notebook uses the `CompVis/stable-diffusion-v1-4` checkpoint as the Stable Diffusion model to fine-tune. However, you are totally free to use any Stable Diffusion checkpoint that you want - you'll just have to adjust the code to load the appropriate components and the safety checker (if it exists). Some interesting models to fine-tune include:

- `runwayml/stable-diffusion-v1-5`
- `prompthero/openjourney`
- `stabilityai/stable-diffusion-2`
- `hakurei/waifu-diffusion`
- `stabilityai/stable-diffusion-2-1`
- `nitrosocke/elden-ring-diffusion`

Evaluation & Leaderboard

To be in the running for the prizes, push one or more DreamBooth models to the Hub with the `dreambooth-hackathon` tag in the model card ([example](#)). This is created automatically by the [DreamBooth notebook](#), but you'll need to add it if you're running your own scripts.

Models are evaluated according to the number of likes they have and you can track your model's ranking on the hackathon's leaderboard:

- [DreamBooth Leaderboard](#)

Timeline

- **December 21, 2022** - Start date
- **December 31, 2022** - Colab Pro registration deadline
- **January 22, 2023** - Final submissions deadline (closing of the leaderboard)
- **January 23-27, 2023** - Announce winners of each theme

All deadlines are at 11:59 PM UTC on the corresponding day unless otherwise noted.

Prizes

We will be awarding 3 prizes per theme, where **winners are determined by the models with the most likes** on the leaderboard:

1st place winner

- [Hugging Face Pro subscription](#) for 1 year or a \$100 voucher for the [Hugging Face merch store](#)

2nd place winner

- A copy of the [NLP with Transformers](#) book or a \$50 voucher for the [Hugging Face merch store](#)

3rd place winner

- [Hugging Face Pro subscription](#) for 1 month or a \$15 voucher for the [Hugging Face merch store](#)

We will also provide a **certificate of completion** to all the participants that submit at least 1 DreamBooth model to the hackathon 🔥.

Compute

Google Colab will be sponsoring this event by providing free Colab Pro credits to 100 participants (selected randomly). We'll be giving out the credits in January 2023, and you have until December 31 to register. To register for these credits, please fill out [this form](#).



FAQ

What data is allowed for fine-tuning?

You can use any images that belong to you or for which a permissive license allows for. If you'd like to submit a model trained on faces (e.g. as a Wildcard submission), we recommend using your own likeness. Ideally, use your own data where you can - we'd love to

see your pets or favorite local landscape features, and we suspect the likes and prizes will tend to go to those who do something nice and personal 😊.

Are other fine-tuning techniques like Textual Inversion allowed?

Absolutely! Although this hackathon is focused on DreamBooth, you're welcome (and encouraged) to experiment with other fine-tuning techniques. This also means you can use whatever frameworks, code, or services that help you make delightful models for the community to enjoy 😊.

Generated from Hugging Face Learn — github.com/huggingface/diffusion-models-class